Logic Design and Switching Theory

Lecture – 2

Subtitle: Logic Gates & Boolean Algebra

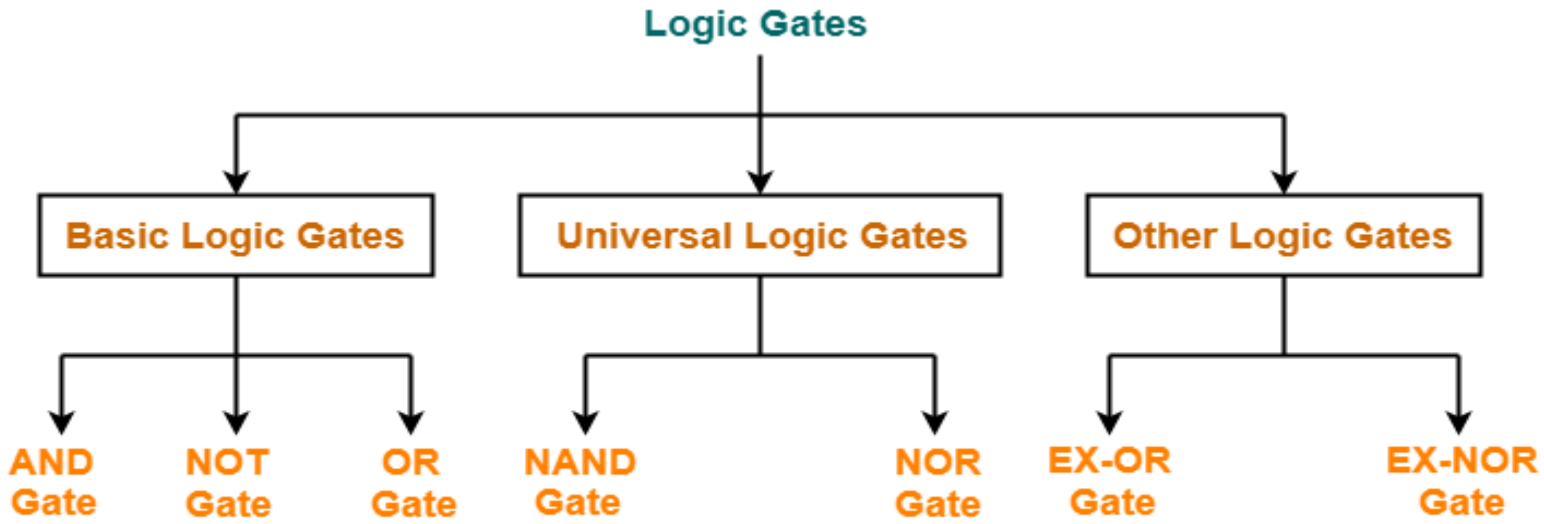
by
Ms. Hera Noor

LOs – Learning Objectives

- ✓**Digital Circuit Building Block**
  - Logic Gates
- ✓**Boolean Algebra**
  - Axioms and Postulates
- ✓**Gray Code**
- ✓**Self Complementing codes**
- ✓**Minterm and Maxterm**
- ✓**K-Map**
- ✓**Constructing logic circuits from equations and truth table**
- ✓**Constructing equations from logic circuits**
- ✓**Problems**

Understanding Digital Circuit Building Block

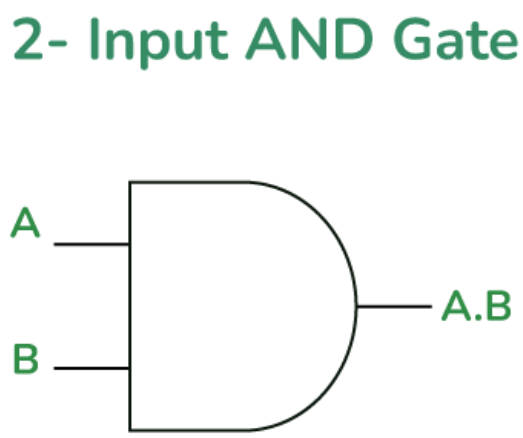
- Logic gates are basic building blocks of digital circuits. They perform logical operations on one or more binary inputs to produce a single binary output.
- Their name is derived from their ability to make decision in the sense that it produces one output when some combination of input levels are given and a different output when other combinations are present.



Types of Logic Gates

AND GATE

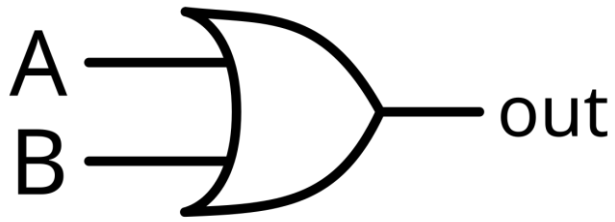
- **Function:** Outputs HIGH (1) only when all inputs are HIGH (1).
- Truth table: the table shows all the input output possibilities.



Input A	Input B	Output (A ∧ B)
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate

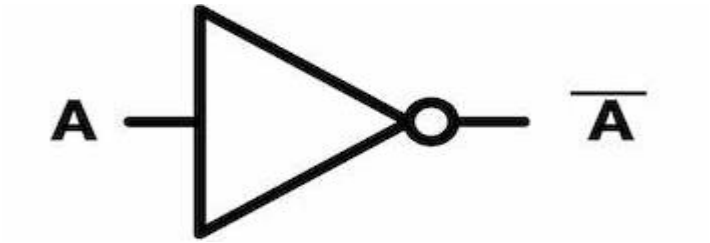
- **Function:** Outputs HIGH (1) if at least one input is HIGH (1).
- Truth table:



Input A	Input B	Output (A ∨ B)
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate (Inverter)

- Has only one input and one output
- **Function:** Outputs the inverse of the input.



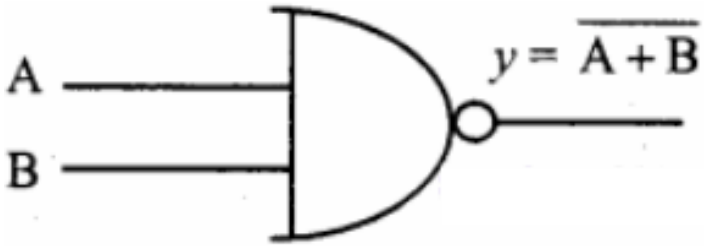
Input	Output (¬A)
0	1
1	0

DERIVED GATES

- Derived gates, also known as special gates, are logic gates that are created from basic gates like AND, OR, and NOT. They have unique symbols, truth tables, and Boolean expressions.

NAND Gate

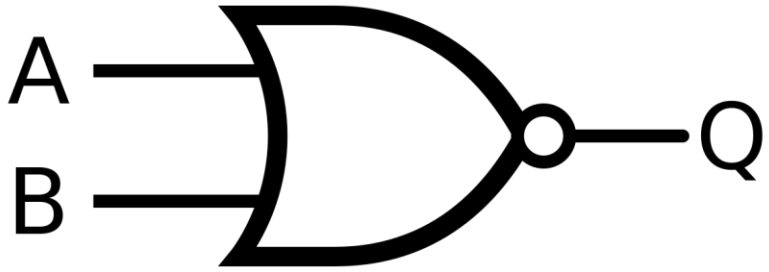
- AND gate followed by an inverter.
- **Function:** Outputs LOW (0) only when all inputs are HIGH (1). It is the inverse of the AND gate.



Input A	Input B	Output (¬(A ∧ B))
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate

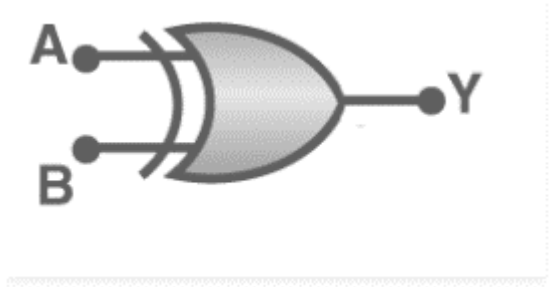
- OR gate followed by an inverter.
- This gate is the combination of OR and NOT gates.
- **Function:** Outputs HIGH (1) only when all inputs are LOW (0). It is the inverse of the OR gate.



Input A	Input B	Output (¬(A ∨ B))
0	0	1
0	1	0
1	0	0
1	1	0

XOR Gate (Exclusive OR)

- **Function:** Outputs HIGH (1) if inputs are different.
- In general, XOR gate is an odd function, output is 1 if the input have odd numbers of 1s.

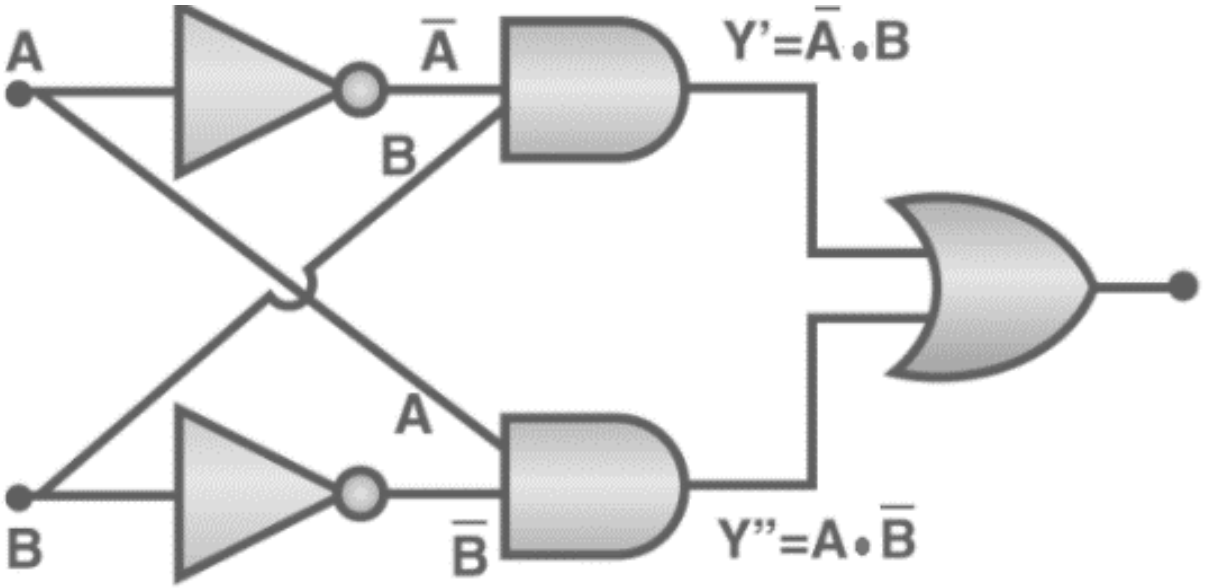


$$Y = A \oplus B$$

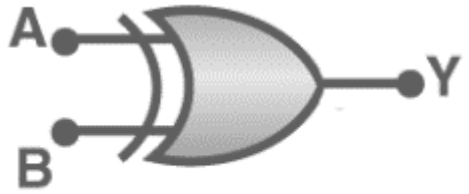
Truth Table

A (Input 1)	B (Input 2)	X = A'B + AB'
0	0	0
0	1	1
1	0	1
1	1	0

XOR Gate (Exclusive OR)



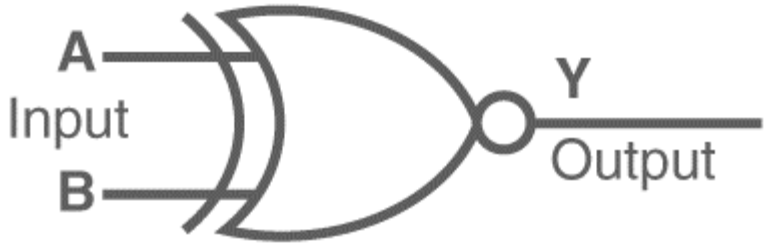
$$A.\bar{B} + \bar{A}.B$$



$$Y = A \oplus B$$

XNOR Gate (Exclusive NOR)

- Output is 1 if both input are same, i.e. if both inputs coincide, also called **coincidence gate**
- **Function:** Outputs HIGH (1) if inputs are the same. It is the inverse of the XOR gate.



Input A	Input B	Output (¬(A ⊕ B))
0	0	1
0	1	0
1	0	0
1	1	1

The Boolean expression of the XNOR gate $Y = (\overline{A \oplus B}) = (A.B + \overline{A}.\overline{B})$

Truth table

A	B	C	XOR (A ⊕ B ⊕ C)	XNOR
0	0	0	0	1
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	0

Applications of Logic Gates

- Basic logic gates are often found in circuits such as safety thermostats, push-button locks, automatic watering systems, light-activated burglar alarms and many other electronic devices.
- **Computers and Processors:** Logic gates are fundamental in designing CPUs and GPUs.
- **Arithmetic Circuits:** Used in adders, subtractors, and multipliers.
- **Control Systems:** Found in automated devices and decision-making circuits.
- **Memory Devices:** Basis for flip-flops and storage components.

Boolean Algebra & Rules

- In 1854, George Boole developed an algebraic system now called Boolean algebra.

	AND Operation	OR Operation	NOT Operation
Axiom1 :	0.0=0	0+0=0	$\bar{0}$=1
Axiom2:	0.1=0	0+1=1	$\bar{1}$=0
Axiom3:	1.0=0	1+0=1	
Axiom4:	1.1=1	1+1=1	

Boolean Algebra & Rules

Complementation law

Law1: $\overline{0}=1$

Law2: $\overline{1}=0$

Law3: if A=0,then $\overline{A}=1$

Law4: if A=1,then $\bar{A}=0$

Law5: if $\overline{\overline{A}}=A$ (double inversion law)

AND Law

Law1: A.0=0 (Null law)

Law2: A.1=A (Identity law)

Law3: A.A=A (Impotence law)

Law4: A. $\overline{A}=0$

OR Law

Law1: A+0=A

Law2: A+1=1

Law3: A+A=A (Impotence law)

Law4: A+ $\overline{A}=1$

Basic Theorems and Laws of Boolean Algebra

Commutative law

Law1: A+B=B+A Law2: A.B=B.A

Associative law

Law1: A + (B +C) = (A +B) +C Law2: A(B.C) = (A.B)C

Distributive law

Law1: A.(B + C) = AB+ AC Law2: A + BC = (A + B).(A +C)

Absorption law

Law1: A +AB =A Law2: A(A +B) = A

De Morgan's Theorem

- **The first theorem** – It states that the NAND gate is equivalent to a bubbled OR gate.

$$\overline{(A \cdot B)} = \overline{A} + \overline{B}$$

- **The second theorem** – It states that the NOR gate is equivalent to a bubbled AND gate.

$$\overline{(A + B)} = \overline{A} \cdot \overline{B}$$

Basic Theorems and Laws of Boolean Algebra

Redundant Literal Rule

Rule1: $A+\overline{A}.B=A+B$

Solution: $A+\overline{A}.B$

$(A+\overline{A}).(A+B)$ $\quad \therefore A + BC = (A + B).(A + C)$

$A+B$ $\quad \therefore A+\overline{A}=1$

Rule2: $A.(\overline{A}+B)=AB$

Solution: $A.(\overline{A}+B)$

$A.\overline{A}+A.B$

AB

Basic Theorems and Laws of Boolean Algebra

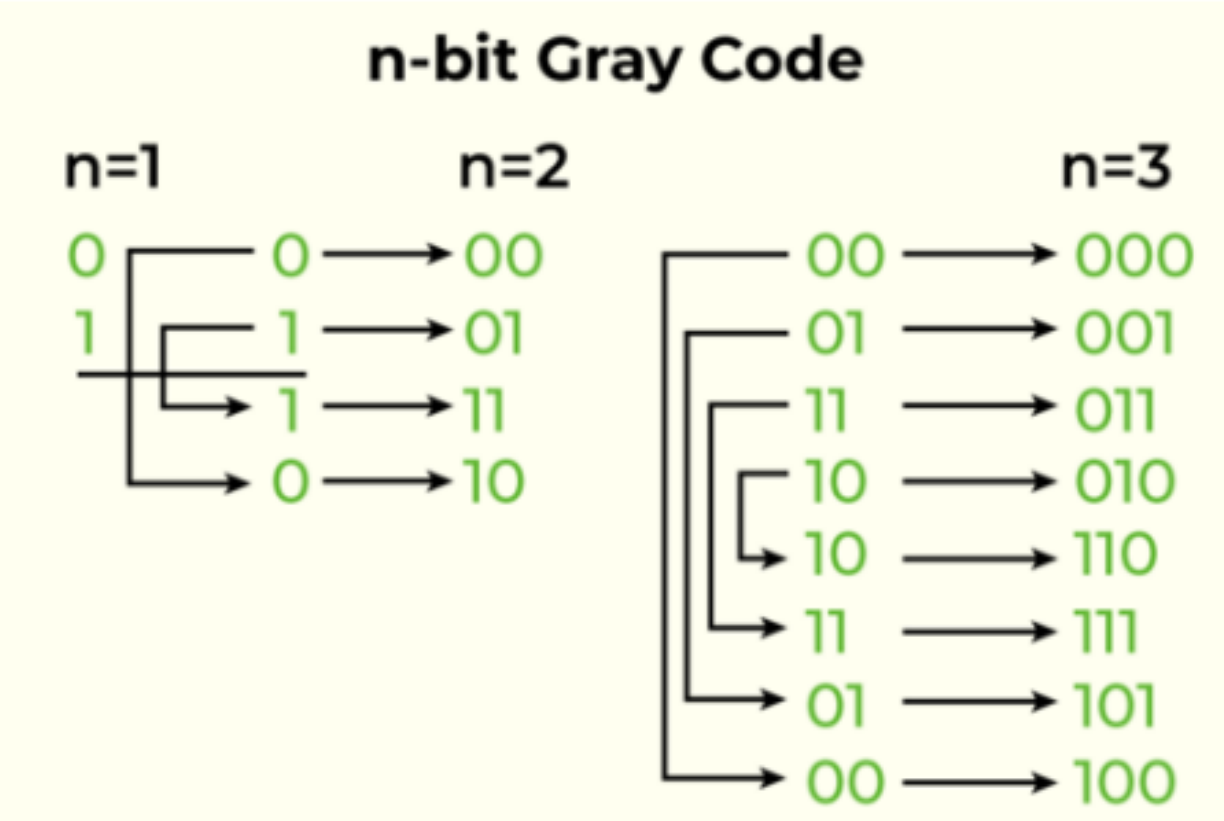
1. **Identity Laws:**
   - $A + 0 = A$
   - $A \cdot 1 = A$
2. **Null Laws:**
   - $A + 1 = 1$
   - $A \cdot 0 = 0$
3. **Idempotent Laws:**
   - $A + A = A$
   - $A \cdot A = A$
4. **Complement Laws:**
   - $A + A' = 1$
   - $A \cdot A' = 0$
5. **Double Negation Law:**
   - $(A')' = A$
6. **Distributive Laws:**
   - $A \cdot (B + C) = (A \cdot B) + (A \cdot C)$
   - $A + (B \cdot C) = (A + B) \cdot (A + C)$
7. **Absorption Laws:**
   - $A + (A \cdot B) = A$
   - $A \cdot (A + B) = A$
8. **De Morgan's Theorems:**
   - $(A \cdot B)' = A' + B'$
   - $(A + B)' = A' \cdot B'$
9. **Consensus Theorem:**
   - $(A + B)(A + B') = A + B$
   - $(A \cdot B) + (A \cdot B') = A$
10. **Universal Bound Laws**
    - $A + A'B = A + B$
    - $A \cdot (A' + B) = A \cdot B$
11. **Redundancy (Covering) Theorem**
    - $A + AB = A$
    - $A(A + B) = A$

Basic Theorems and Laws of Boolean Algebra

Part-A	Part-B
A+0=A	A.0=0
A+1=1	A.1=A
A+A=A (Impotence law)	A.A=A (Impotence law)
$A+\overline{A}=1$	$A.\overline{A}=0$
$\overline{\overline{A}}=A$ (double inversion law)	--
Commutative law: A+B=B+A	A.B=B.A
Associative law: A + (B +C) = (A +B) +C	A(B.C) = (A.B)C
Distributive law: A.(B + C) = AB+ AC	A + BC = (A + B).(A +C)
Absorption law: A +AB =A	A(A +B) = A
DeMorgan Theorem: $\overline{(A + B)}=\overline{A}.\overline{B}$	$\overline{(A . B)}=\overline{A}+\overline{B}$
Redundant Literal Rule: $A+\overline{A}.B=A+B$	$A.(\overline{A}+B)=AB$

Gray code

Gray code is a unit distance code, also known as Reflected Binary Code, is a binary numeral system in which two successive values differ in only one bit. This is in contrast to standard binary representation, where multiple bits might change when transitioning from one value to the next.



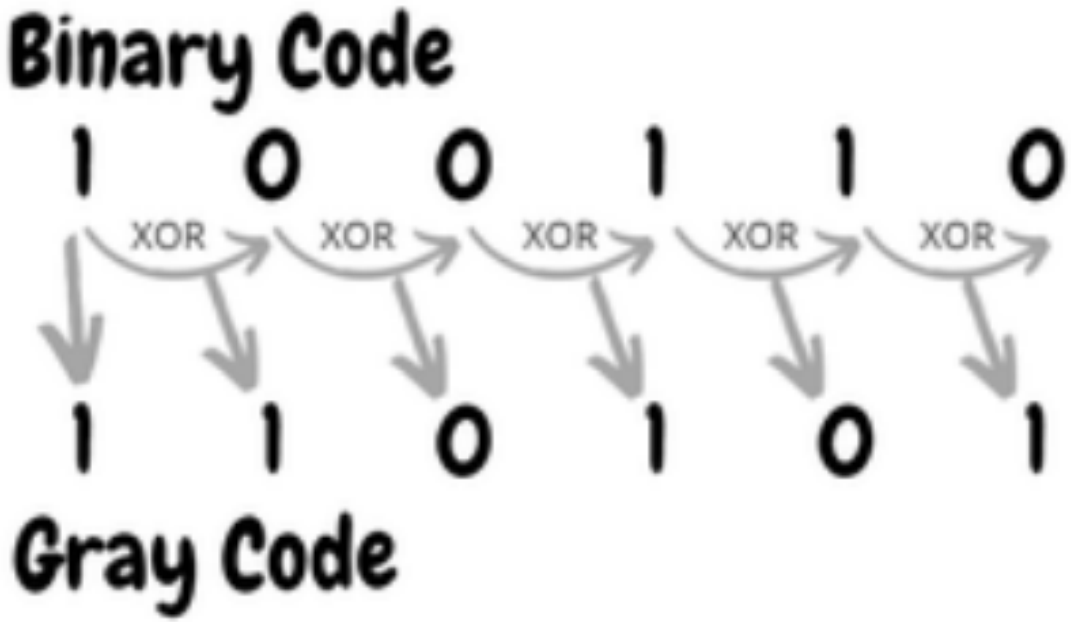
Decimal	*Binary Code*	*Gray Code*	*Decimal*	*Binary Code*	*Gray Code*
0	0000	0000	8	1000	1100
1	0001	0001	9	1001	1101
2	0010	0011	10	1010	1111
3	0011	0010	11	1011	1110
4	0100	0110	12	1100	1010
5	0101	0111	13	1101	1011
6	0110	0101	14	1110	1001
7	0111	0100	15	1111	1000

Binary to Gray Code

The most significant bit (MSB) of the binary number is the same as the MSB o the Gray code.

For each subsequent bit:

Binary bit = Previous Binary bit XOR Current Gray bit.

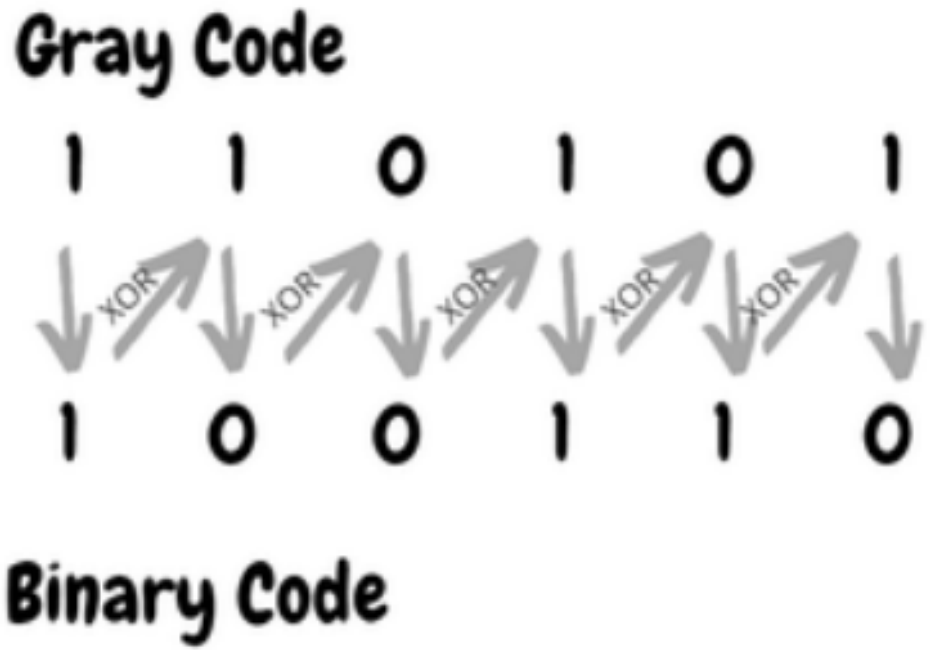


Gray Code to Binary

The MSB of the Binary number is the same as the MSB of the Gray code.

Other Bits:

For each subsequent bit, **XOR the previous Binary bit with the corresponding Gray bit.**



Self complementing code

Self-complementing codes are special codes in which the 1's complement of a valid codeword represents the complement of the decimal number it encodes. These codes are widely used in error detection and digital systems.

- The code is self-inverse: The complement of a codeword corresponds to the 9's complement of the decimal number.
- The sum of the digit weights in any decimal number representation equals 9. This ensures self-complementarity.
- These codes simplify the computation of complements for error detection and correction.

Some common Self complementing code

1. **Excess-3 Code**

- Each decimal digit is represented in binary form after adding $+3$.
- Example:
  - $0 \rightarrow 0011$ (3 added to 0)
  - $1 \rightarrow 0100$ (3 added to 1)
  - $9 \rightarrow 1100$ (3 added to 9)

Complement Property:

- The complement of 0011 (for 0) is 1100, which represents 9.

Some common Self complementing code

2. 9's Complement Code

- Each decimal digit d is encoded as $9 - d$.
- Example:
  - $0 \rightarrow 1001$ (9 - 0)
  - $1 \rightarrow 1000$ (9 - 1)
  - $9 \rightarrow 0000$ (9 - 9)

Complement Property:

- 1001 for 0 complements 0000 for 9.

Some common Self complementing code

3. **2-4-2-1 Code**

- Weighted binary code with weights $2, 4, 2, 1$.
- Example:
  - $0 \rightarrow 0000$
  - $1 \rightarrow 0001$
  - $9 \rightarrow 1001$

Complement Property:

- The code's weights ensure self-complementarity.

Some common Self complementing code

Decimal digit	BCD(8421)	2421	Excess-3
0	0000	0000	0011
1	0001	0001	0100
2	0010	0010	0101
3	0011	0011	0110
4	0100	0100	0111
5	0101	1011	1000
6	0110	1100	1001
7	0111	1101	1010
8	1000	1110	1011
9	1001	1111	1100

Minterms and Maxterms

Minterms

It is known as the product term. In the minterm, each uncomplemented term is indicated by ‘1’, and each complemented term is indicated by ‘0’.

Example 011= A’BC

Maxterms

It is known as the sum term. In maxterm, each uncomplemented term is indicated by ‘0’ and each complemented term is indicated by ‘1’.

Example: – 100= A’+B+C

Karnaugh maps or K – maps

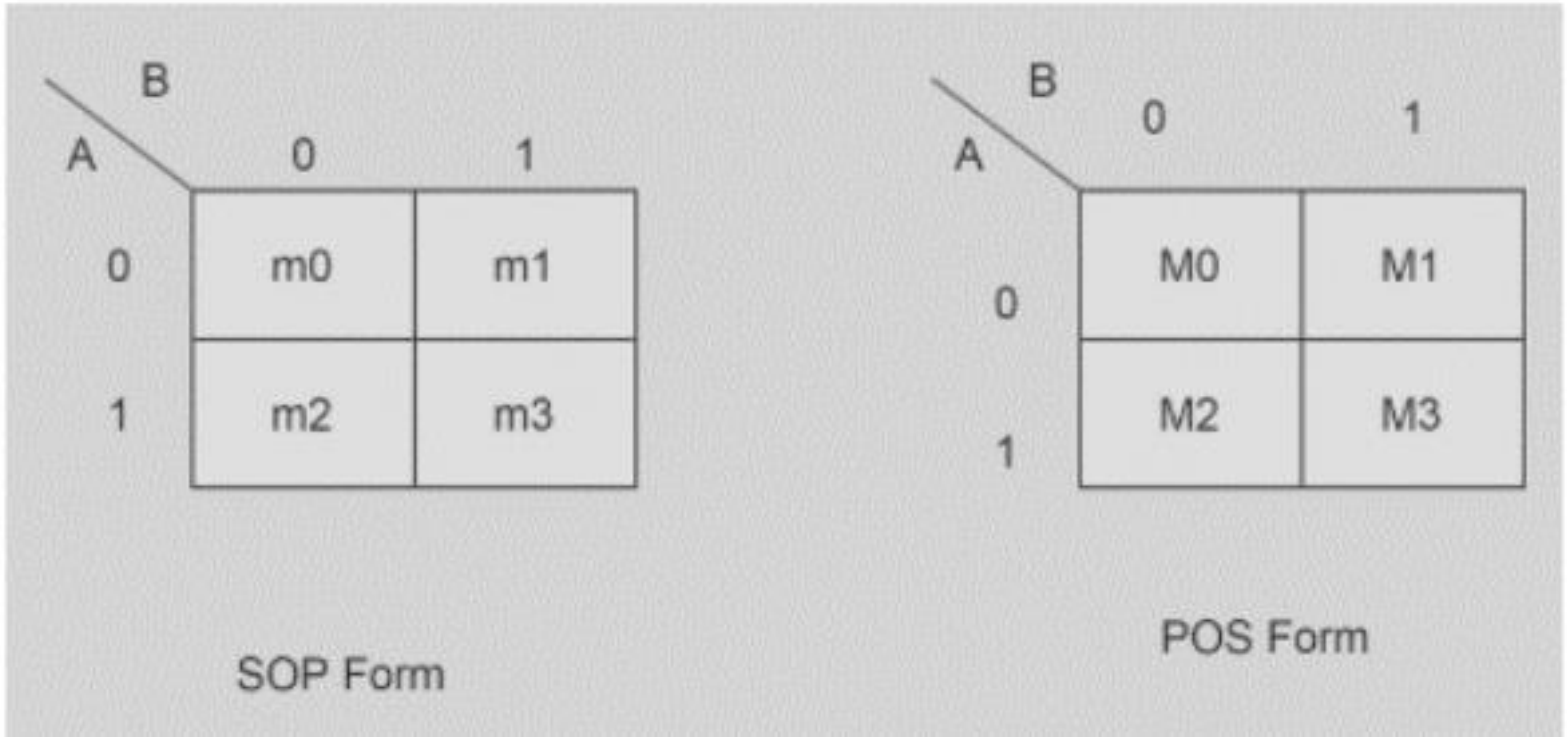
Karnaugh maps are used to simplify real-world logic requirements so that they can be implemented using the minimal number of logic gates.

A sum-of-products expression (SOP) can always be implemented using AND gates feeding into an OR gate, a product-of-sums expression (POS) leads to OR gates feeding an AND gate.

This approach may be applied for any number generally it is used up to six variables, exceeding which it becomes unmanageable. For an n variable K-map, there are 2n cells. In K-map, Gray code is applied for the recognition of cells.

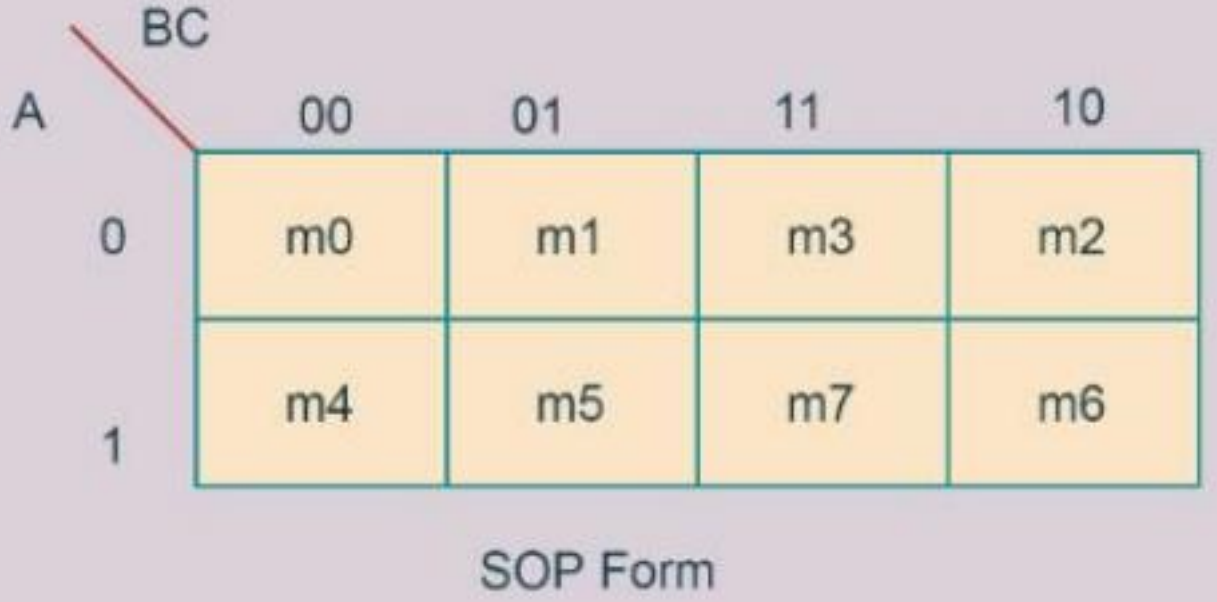
Two Variable K-Map

The number of cells in a two-variable K-map is four as the number of variables is two, i.e. a two-variable K-map will have either four minterms or four maxterms. The below figure shows two variable K-Map with SOP and POS representation.

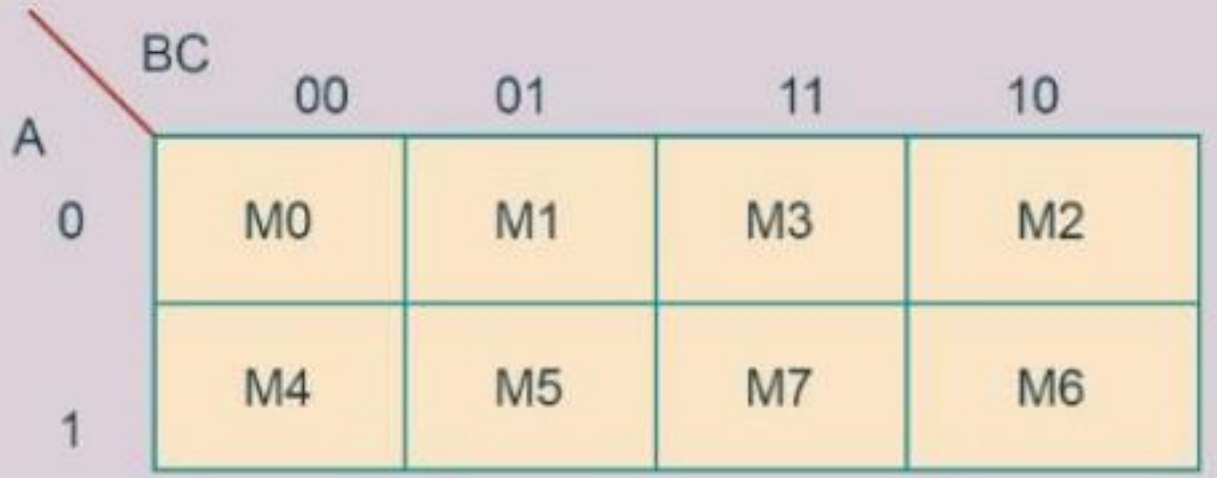


Three-Variable K-map

The three-variable K-map is represented by an array of eight cells(eight minterms or eight maxterms). For understanding purposes, we will use A, B, and C as the three variables. However one can use any letter for the names of the variables. The below figure presents three variables K-Map with SOP and POS representation.



SOP Form



POS Form

Four-Variable K-map

The four-variable K-map is expressed as an array of 16 cells i.e. sixteen minterms or sixteen maxterms. The below figure presents a four-variable K-Map with SOP and POS representation.

Problems!

- Air conditioning unit is to be turned on (by typical one at its control unit). If the temperature of the any of the three locations exceeds 78 F assuming that the temperature sensing devices at the locations each produce a logical one when the temperature exceed and a logical zero otherwise. Design a logical required to control the air conditioning.

- Design a logic expression that equals one only when the two binary number A1.A0 and B1.B0 have the same value. Draw the logic diagram and construct the table to verify logic.

Problems!

- Let sunshine be zero. If the sun is shining and 1 if it is not. Let rain be zero if it is raining and 1 if it is not. Draw the logic diagram for rainbow which is 1 only if the sun is shining and it is not raining.
- A logic circuit must produce zero when the binary three binary numbers has its maximum possible values. Draw the logic diagram, truth table and give its Boolean equation.
- An alarm buzzer is to sound when the ignition is turned on and the gear shift is engaged provided that either the front seat is occupied and the seat belt is not fastened. Find the logic expression and draw diagram.

Problems!

- Develop the logic expressions for the following scenarios.

i. You will gain weight if you eat too much or you do not exercise enough and your metabolism rate is too low.
ii. The roads will be slippery if it snows or it rains, and there is oil on the road.
iii. The buzzer will sound if the key is in the ignition switch or the car door is open or the seat belts are not fastened.

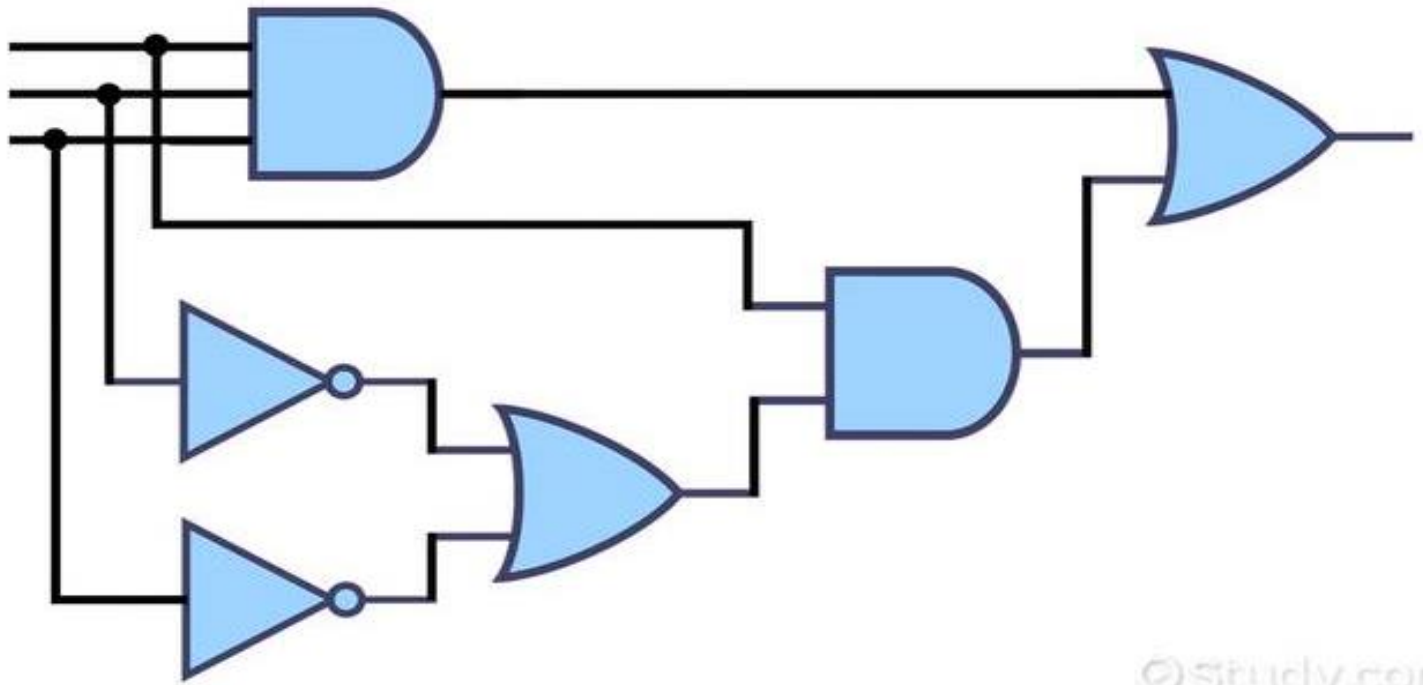
Some Key points

- **Fundamental logical functions are performed using basic logic gates. These are the fundamental components of integrated circuits.**
- **AND gate, OR gate, XOR gate, NAND gate, NOR gate, XNOR gate and NOT gate are the seven types of basic logic gates.**
- **A universal gate is a logic gate that can implement any Boolean function without using another logic gate. The universal gates are the NOR and NAND gates.**

Some Key points

- **Logic gates are essential components in digital electronics.**
- **They perform logical operations based on binary inputs.**
- **Widely used in computing, automation, and communication technologies.**

Construct Boolean Equation and Truth table to verify it



Construct Logic Diagram and Truth table to verify it

1) $F(A,B,C) = A \cdot (\overline{B+C}) + \overline{A} \cdot B$

2) $F(W,X,Y,Z) = (W \cdot X) + (\overline{Y} \cdot Z)$

3) $F(P,Q,R) = (\overline{P}+Q) \cdot (P+\overline{R})$

4) $F(A,B,C,D) = \overline{A \cdot B} + (\overline{C}+D) \cdot B$

5) $F(X,Y,Z) = (X+\overline{Y}) \cdot (Y+Z) \cdot \overline{X \cdot Z}$

Thank you!